

AI Agent 记忆系统：从短期到长期的技术架构与实践

mp.weixin.qq.com/s/AeqetxatPrZXapanYWRC6g

前言

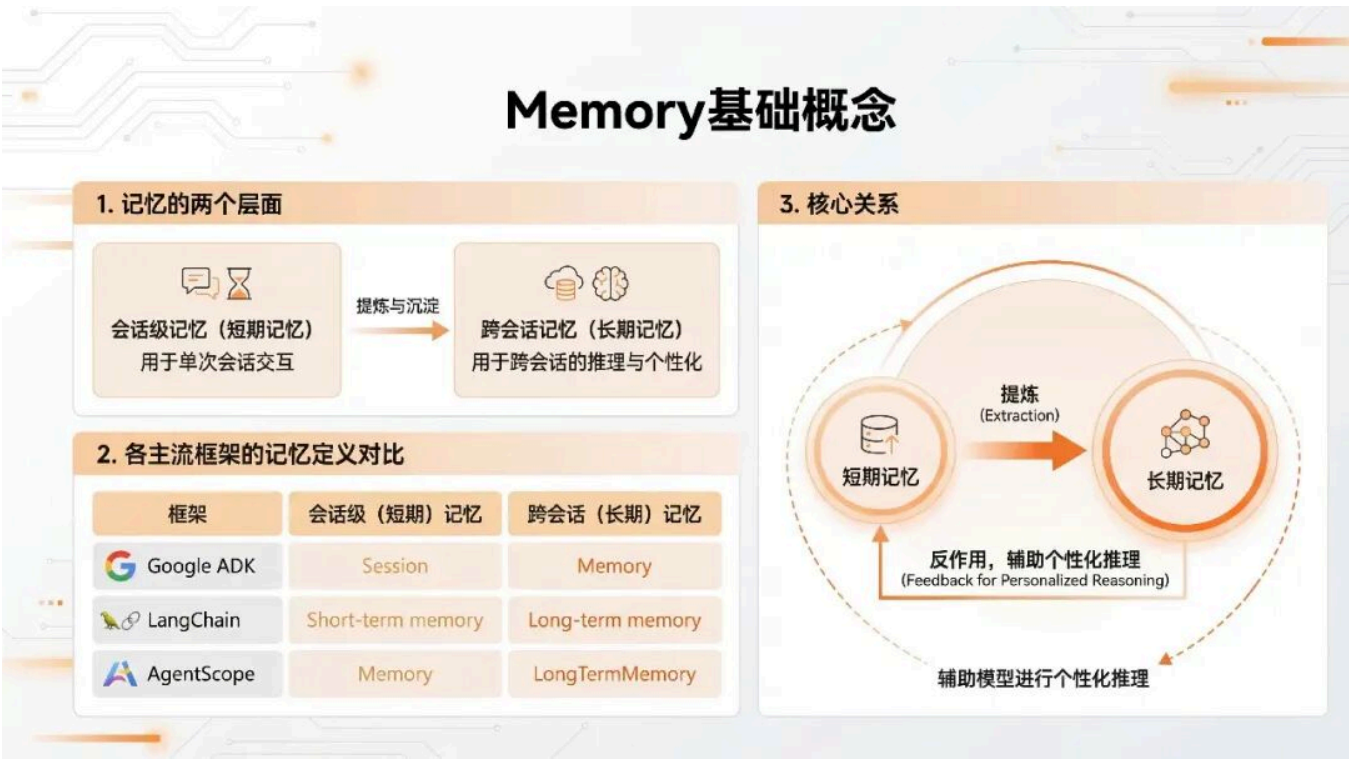
Cloud Native

随着 AI Agent 应用的快速发展，智能体需要处理越来越复杂的任务和更长的对话历史。然而，LLM 的上下文窗口限制、不断增长的 token 成本，以及如何让 AI“记住”用户偏好和历史交互，都成为了构建实用 AI Agent 系统面临的核心挑战。记忆系统（Memory System）正是为了解决这些问题而诞生的关键技术。

记忆系统使 AI Agent 能够像人类一样，在单次对话中保持上下文连贯性（短期记忆），同时能够跨会话记住用户偏好、历史交互和领域知识（长期记忆）。这不仅提升了用户体验的连续性和个性化程度，也为构建更智能、更实用的 AI 应用奠定了基础。

01 Memory 基础概念

Cloud Native



1.1 记忆的定义与分类

对于 AI Agent 而言，记忆至关重要，因为它使它们能够记住之前的互动、从反馈中学习，并适应用户的偏好。

对“记忆”的定义有两个层面：

- 会话级记忆：用户和智能体 Agent 在一个会话中的多轮交互（user-query & response）
- 跨会话记忆：从用户和智能体 Agent 的多个会话中抽取的通用信息，可以跨会话辅助 Agent 推理

1.2 各 Agent 框架的定义差异

各个 Agent 框架对记忆的概念命名各有不同，但共同的是都遵循上一节中介绍的两个不同层面的划分：会话级和跨会话级。

框架	会话级记忆	跨会话记忆	备注
Google ADK	Session	Memory (Long-Term Knowledge)	Memory作为可搜索的知识库
LangChain	Short-term memory	Long-term memory	Long-term memory属于高阶辅助组件
AgentScope	Memory	LongTermMemory	API层面仍为两个组件，功能有明确区分

框架说明：

- Google ADK：Session 表示单次持续交互；Memory 是长期知识库，可包含来自多次对话的信息
- LangChain：Short-term memory 用于单线程或对话中记住之前的交互；Long-term memory 不属于基础核心组件，而是高阶的“个人知识库”外挂
- AgentScope：虽然官方文档强调需求驱动，但 API 层面仍然是两个组件（memory 和 long_term_memory），功能层面有明确区分

习惯上，可以将会话级别的历史消息称为**短期记忆**，把可以跨会话共享的信息称为**长期记忆**，但本质上两者并不是通过简单的时间维度进行的划分，从实践层面上以是否跨 Session 会话来进行区分。长期记忆的信息从短期记忆中抽取提炼而来，根据短期记忆中的信息实时地更新迭代，而其信息又会参与到短期记忆中辅助模型进行个性化推理。

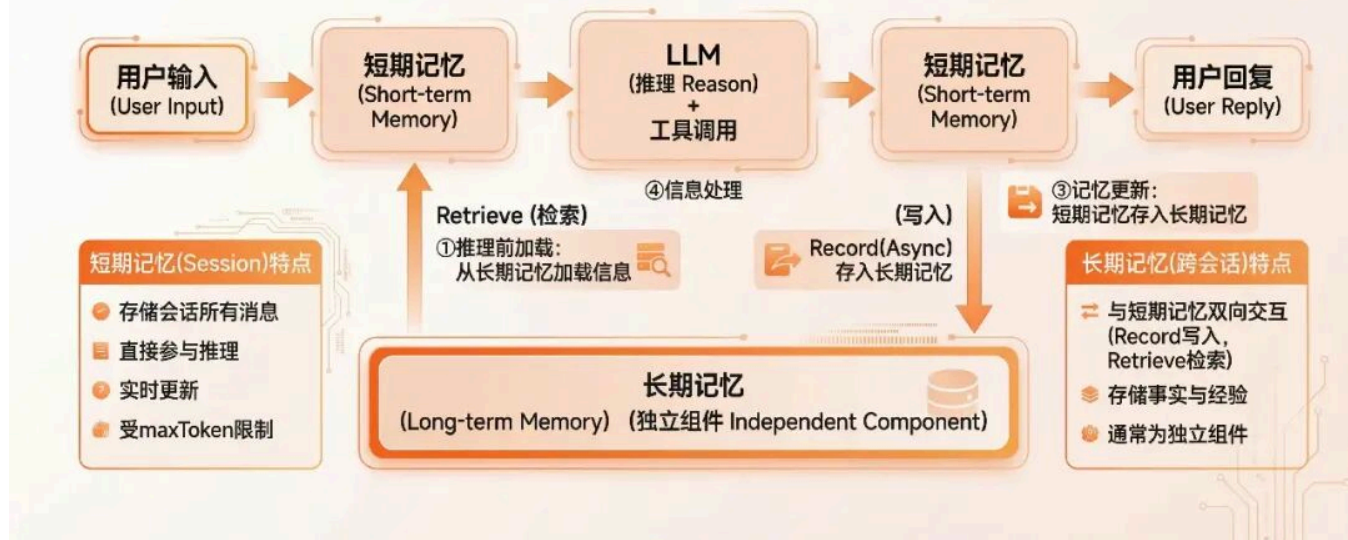
02
Agent 框架集成记忆系统的架构

Cloud Native

各 Agent 框架在集成记忆系统时，虽然实现细节不同，但都遵循相似的架构模式。理解这些通用模式有助于更好地设计和实现记忆系统。

2.1 Agent 框架集成记忆的通用模式

Agent框架集成记忆的通用模式



各 Agent 框架集成记忆系统通常遵循以下通用模式：

1. Step1：推理前加载 - 根据当前 user-query 从长期记忆中加载相关信息
2. Step2：上下文注入 - 从长期记忆中检索的信息加入当前短期记忆中辅助模型推理
3. Step3：记忆更新 - 短期记忆在推理完成后加入到长期记忆中
4. Step4：信息处理 - 长期记忆模块中结合 LLM+向量化模型进行信息提取和检索

2.2 短期记忆（Session 会话）

短期记忆存储会话中产生的各类消息，包括用户输入、模型回复、工具调用及其结果等。这些消息直接参与模型推理，实时更新，并受模型的 maxToken 限制。当消息累积导致上下文窗口超出限制时，需要通过上下文工程策略（压缩、卸载、摘要等）进行处理，这也是上下文工程主要处理的部分。

核心特点：

- 存储会话中的所有交互消息（用户输入、模型回复、工具调用等）
- 直接参与模型推理，作为 LLM 的输入上下文
- 实时更新，每次交互都会新增消息
- 受模型 maxToken 限制，需要上下文工程策略进行优化

关于短期记忆的上下文工程策略（压缩、卸载、摘要等），将在下一章节中详细介绍。

2.3 长期记忆（跨会话）

长期记忆与短期记忆形成双向交互：一方面，长期记忆从短期记忆中提取“事实”、“偏好”、“经验”等有效信息进行存储（Record）；另一方面，长期记忆中的信息会被检索并注入到短期记忆中，辅助模型进行个性化推理（Retrieve）。

与短期记忆的交互：

- **Record (写入)**：从短期记忆的会话消息中提取有效信息，通过LLM进行语义理解和抽取，存储到长期记忆中
- **Retrieve (检索)**：根据当前用户查询，从长期记忆中检索相关信息，注入到短期记忆中作为上下文，辅助模型推理

实践中的实现方式：

在 Agent 开发实践中，长期记忆通常是一个独立的第三方组件，因为其内部有相对比较复杂的流程（信息提取、向量化、存储、检索等）。常见的长期记忆组件包括 Mem0、Zep、Memos、ReMe 等，这些组件提供了完整的 Record 和 Retrieve 能力，Agent 框架通过 API 集成这些组件。

信息组织维度：

不同长期记忆产品在信息组织维度上有所差异：一些产品主要关注个人信息（个人记忆），而一些产品除了支持个人记忆外，还支持工具记忆、任务记忆等更丰富的维度。

1. 用户维度（个人记忆）：面向用户维度组织的实时更新的个人知识库

- 用户画像分析报告
- 个性化推荐系统，千人千面
- 处理具体任务时加载至短期记忆中

2. 业务领域维度：沉淀的经验（包括领域经验和工具使用经验）

- 可沉淀至领域知识库
- 可通过强化学习微调沉淀至模型

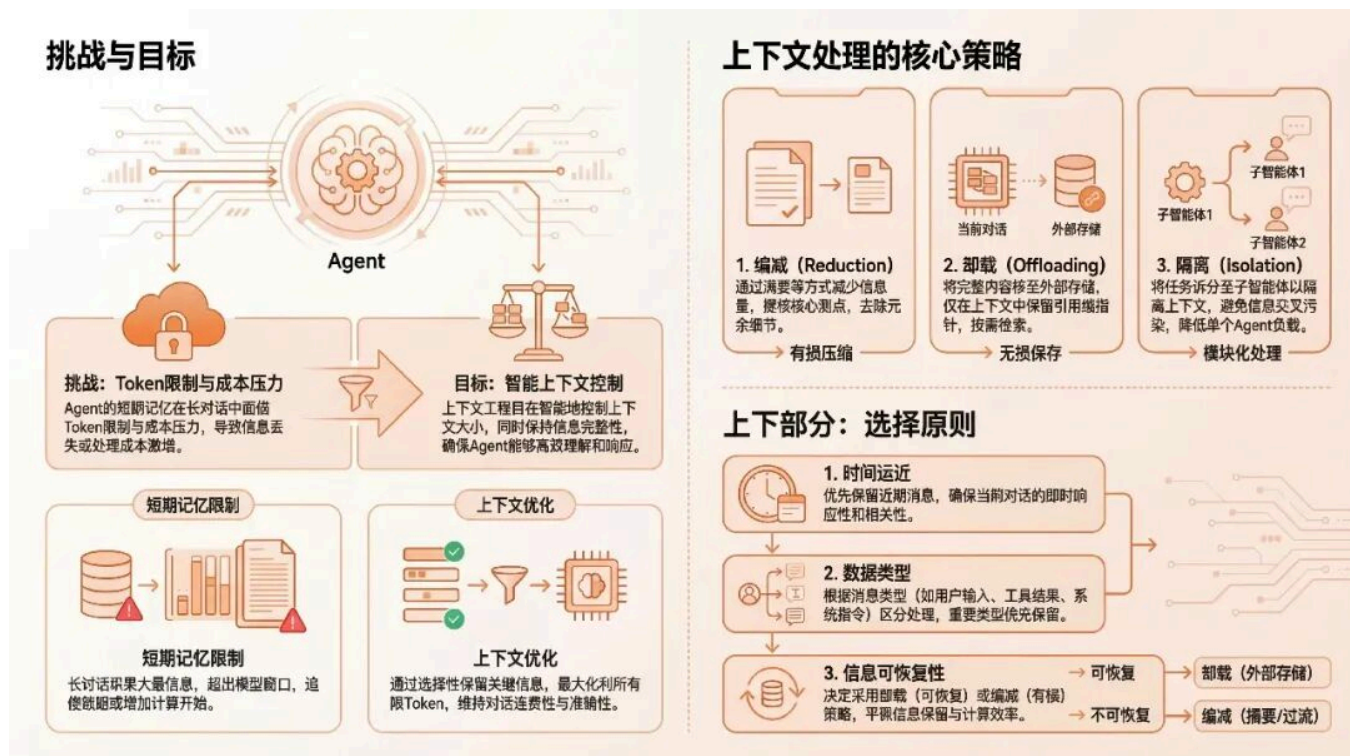
03

短期记忆的上下文工程策略

Cloud Native

短期记忆直接参与 Agent 和 LLM 的交互，随着对话历史增长，上下文窗口会面临 token 限制和成本压力。上下文工程策略旨在通过智能化的压缩、卸载和摘要技术，在保持信息完整性的同时，有效控制上下文大小。

备注：需要说明的是，各方对上下文工程的概念和理解存在些许差异。**狭义的上下文工程**特指对短期记忆（会话历史）中各种压缩、摘要、卸载等处理机制，主要解决上下文窗口限制和 token 成本问题；**广义的上下文工程**则包括更广泛的上下文优化策略，如非运行态的模型选择、Prompt 优化工程、知识库构建、工具集构建等，这些都是在模型推理前对上下文进行优化的手段，且这些因素都对模型推理结果有重要影响。本章节主要讨论狭义的上下文工程，即针对短期记忆的运行时处理策略。



3.1 核心策略

针对短期记忆的上下文处理，主要有以下几种策略：

上下文缩减 (Context Reduction)

上下文缩减通过减少上下文中的信息量来降低 token 消耗，主要有两种方法：

- 1. 保留预览内容：**对于大块内容，只保留前 N 个字符或关键片段作为预览，原始完整内容被移除
 - 2. 总结摘要：**使用 LLM 对整段内容进行总结摘要，保留关键信息，丢弃细节
- 这两种方法都会导致信息丢失，但能有效减少 token 消耗。

上下文卸载 (Context Offloading)

上下文卸载主要解决被缩减的内容是否可恢复的问题。当内容被缩减后，原始完整内容被卸载到外部存储（如文件系统、数据库等），消息中只保留最小必要的引用（如文件路径、UUID 等）。当需要完整内容时，可以通过引用重新加载。

优势：上下文更干净，占用更小，信息不丢，随取随用。适用于网页搜索结果、超长工具输出、临时计划等占 token 较多的内容。

上下文隔离 (Context Isolation)

通过多智能体架构，将上下文拆分到不同的子智能体中（类似单体拆分称多个微服务）。主智能体编写任务指令，发送给子智能体，子智能体的整个上下文仅由该指令组成。子智能体完成任务后返回结果，主智能体不关心子智能体如何执行，只需要结果。

适用场景：任务有清晰简短的指令，只有最终输出才重要，如代码库中搜索特定片段。

优势：上下文小、开销低、简单直接。

策略选择原则：

以上三种策略（上下文缩减、上下文卸载、上下文隔离）需要根据数据的分类进行综合处理，主要考虑因素包括：

- **时间远近：**近期消息通常更重要，需要优先保留；历史消息可以优先进行缩减或卸载
- **数据类型：**不同类型的消息（用户输入、模型回复、工具调用结果等）重要性不同，需要采用不同的处理策略
- **信息可恢复性：**对于需要完整信息的内容，应优先使用卸载策略；对于可以接受信息丢失的内容，可以使用缩减策略

3.2 各框架的实现方式

各框架一般内置上下文处理策略，通过参数化配置的方式指定具体策略。

Google ADK

构建 Agent 时通过 `events_compaction_config` 设置上下文处理策略，和 Session 本身的数据存储独立。

```
from google.adk.apps.app import App, EventsCompactionConfig
app = App(    name='my-agent',    root_agent=root_agent,    events_compaction_config=EventsCompactionConfig(        compaction_interval=3, # 每3次新调用触发压缩        overlap_size=1        # 包含前一个窗口的最后一次调用    ),)
```

LangChain

构建 Agent 时通过 middleware 机制中的 `SummarizationMiddleware` 设置上下文处理参数，与短期记忆本身的数据存储独立。

```
from langchain.agents import create_agent
from langchain.agents.middleware import SummarizationMiddleware
agent = create_agent(    model="gpt-4o",    tools=[...],    middleware=[        SummarizationMiddleware(            model="gpt-4o-mini",            max_tokens_before_summary=4000, # 4000 tokens时触发摘要            messages_to_keep=20, # 摘要后保留最后20条消息        ),    ],)
```

AgentScope

AgentScope 通过 **AutoContextMemory** 提供智能化的上下文工程解决方案。

`AutoContextMemory` 实现了 `Memory` 接口，当对话历史超过配置阈值时，自动应用 6 种渐进式压缩策略（从轻量级到重量级）来减少上下文大小，同时保留重要信息。

集成方式：

- 直接作为 `Memory` 接口实现，通过 `memory` 参数集成到 Agent 中
- 与框架深度集成，无需额外的 middleware 或独立配置

与 ADK 和 LangChain 的差异：

- **更精细化的压缩策略**：提供 6 种渐进式压缩策略（压缩历史工具调用、卸载大型消息、摘要对话轮次等），相比 ADK 的简单压缩和 LangChain 的摘要 middleware，策略更加细化和可控
- **集成方式**：直接实现 Memory 接口，与 Agent 构建流程无缝集成，而 ADK 和 LangChain 需要独立的配置对象或 middleware 机制
- **完整可追溯性**：提供工作内存、原始内存、卸载上下文和压缩事件四层存储架构，支持完整历史追溯，而其他框架通常只提供压缩后的结果

使用示例：

```
AutoContextMemory memory = new AutoContextMemory(      AutoContextConfig.builder()
    .msgThreshold(100)          .maxToken(128 * 1024)      .tokenRatio(0.75)
    .build(),      model); ReActAgent agent = ReActAgent.builder()      .name("Assistant")
    .model(model)      .memory(memory)      .build();
```

详细文档：关于 AutoContextMemory 的 6 种压缩策略、存储架构和高级配置，请参考 [AutoContextMemory 详细文档](#)。

04

长期记忆技术架构及 Agent 框架集成

Cloud Native

与短期记忆不同，长期记忆需要跨会话持久化存储，并支持高效的检索和更新。这需要一套完整的技术架构，包括信息提取、向量化存储、语义检索等核心组件。

4.1 核心组件

长期记忆涉及 record & retrieve 两个核心流程，需要以下核心组件：

1. **LLM 大模型**：提取短期记忆中的有效信息（记忆的语义理解、抽取、决策和生成）
2. **Embedder 向量化**：将文本转换为语义向量，支持相似性计算
3. **VectorStore 向量数据库**：持久化存储记忆向量和元数据，支持高效语义检索
4. **GraphStore 图数据库**：存储实体-关系知识图谱，支持复杂关系推理
5. **Reranker（重排序器）**：对初步检索结果按语义相关性重新排序
6. **SQLite**：记录所有记忆操作的审计日志，支持版本回溯

4.2 Record & Retrieve 流程

Record（记录）

LLM 事实提取 → 信息向量化 → 向量存储 → (复杂关系存储) → SQLite 操作日志

Retrieve（检索）

User query 向量化 → 向量数据库语义检索 → 图数据库关系补充 → (Reranker-LLM) → 结果返回

4.3 长期记忆与 RAG 的区别

像 Mem0 这类面向 AI Agent 的个性化长期记忆系统，与 RAG（Retrieval-Augmented Generation）在技术架构上有诸多相似之处，但功能层面和场景上有明显区别：

对比条目	RAG	长期记忆
主要目的	为大模型提供外部知识，弥补训练数据的局限性（如时效性、专业性）	为 AI Agent 记录并利用特定用户的历史交互信息，实现个性化、上下文连续的服务
服务对象	全体用户或任务	特定用户或会话主体（高度个性化）
知识来源	结构化/非结构化文档（如 PDF、网页、数据库）	用户与 Agent 的对话历史、行为日志等

技术层面的相似点：

- 1. 向量化存储：都将文本内容通过 Embedding 模型转为向量，存入向量数据库
- 2. 相似性检索：在用户提问时，将当前 query 向量化，在向量库中检索 top-k 最相关的条目
- 3. 注入上下文生成：将检索到的内容注入到模型交互上下文中，辅助 LLM 生成最终回答

4.4 关键问题与挑战

长期记忆系统在实际应用中面临诸多挑战，这些挑战直接影响系统的可用性和用户体验。

1. 准确性

记忆的准确性包含两个层面：

- 有效的记忆管理：需要具备智能的巩固、更新和遗忘机制，这主要依赖于记忆系统中负责信息提取的模型能力和算法设计
- 记忆相关性的检索准确度：主要依赖于向量化检索&重排的核心能力

核心挑战：

- 记忆的建模：需要完善强大的用户画像模型
- 记忆的管理：基于用户画像建模算法，提取有效信息，设计记忆更新机制
- 向量化相关性检索能力：提升检索准确率和相关性

2. 安全和隐私

记忆系统记住了大量用户隐私信息，如何防止数据中毒等恶意攻击，并保障用户隐私，是必须解决的问题。

核心挑战：

- 数据加密与访问控制
- 防止恶意数据注入
- 透明的数据管理机制
- 用户对自身数据的掌控权

3. 多模态记忆支持

文本记忆、视觉、语音仍被孤立处理，如何构建统一的“多模态记忆空间”仍是未解难题。

核心挑战：

- 跨模态关联与检索
- 统一的多模态记忆表示
- 毫秒级响应能力

4.5 Agent 框架集成

在 AgentScope 中，可以通过集成第三方长期记忆组件来实现长期记忆功能。常见的集成方式包括：

4.5.1 集成 Mem0

Mem0 是一个开源的长期记忆框架，几乎成为事实标准。在 AgentScope 中集成 Mem0 的示例：

```
// 初始化Mem0长期记忆Mem0LongTermMemory mem0Memory = new Mem0LongTermMemory(
Mem0Config.builder()          .apiKey("your-mem0-api-key")          .build());// 创建Agent并
集成长期记忆ReActAgent agent = ReActAgent.builder()          .name("Assistant")
.model(model)          .memory(memory) // 短期记忆          .longTermMemory(mem0Memory) // 长期记忆
          .build();
```

4.5.2 集成 ReMe

ReMe 是 AgentScope 官方提供的长期记忆实现，与框架深度集成：

```
// 初始化ReMe长期记忆ReMeLongTermMemory remeMemory = ReMeLongTermMemory.builder()
.userId("user123") // 用户ID，用于记忆隔离          .apiBaseUrl("http://localhost:8002") //
ReMe服务地址          .build();// 创建Agent并集成长期记忆ReActAgent agent = ReActAgent.builder()
          .name("Assistant")          .model(model)          .memory(memory) // 短期记忆
          .longTermMemory(remeMemory) // 长期记忆          .longTermMemoryMode(LongTermMemoryMode.BOTH)
// 记忆模式          .build();
```

05

行业趋势与产品对比

Cloud Native

5.1 AI 记忆系统发展趋势

AI 记忆系统的核心目标是让 AI 能像人类一样持续学习、形成长期记忆，从而变得更智能、更个性化。当前行业呈现出从研究原型向生产级系统演进、从单一技术向综合解决方案发展的趋势。

5.1.1 当前发展的核心脉络

趋势维度	核心特点	代表技术与系统
类脑计算	借鉴人脑记忆形成与检索机制，构建分层、动态的记忆架构	EverMemOS的四层架构，Google的Nested Learning
多模态融合	整合文本、图像、音频等多种信息来源，形成统一的世界记忆	多模态智能体，跨模态检索与融合技术
工程化与系统化	研究原型走向稳定、可扩展的生产级系统，提供统一解决方案	EverMemOS, Mem0, O-Mem
智能体能力强化	记忆成为智能体具备长期一致性、主动性和自我演化的基石	应用于对话、游戏、科研等复杂任务的智能体

5.1.2 技术发展趋势

记忆即服务 (Memory-as-a-Service, MaaS)

AI Agent 是大模型、记忆、任务规划以及工具使用的集合体，记忆管理将是 Agent 智能体的核心基础功能之一。类似“数据库”之于传统软件，记忆系统将成为 AI 应用的基础设施，提供标准化的记忆服务接口、可扩展的存储和检索能力。

精细化记忆管理

借鉴人脑记忆机制，构建分层动态的记忆架构，对记忆进行全生命周期管理。技术路径包括：LLM 驱动记忆提取 + 向量化存储 + 图数据库补充；向量化检索（海马体）+ LLM 提纯（大脑皮层）结合；通过强化学习提升记忆管理表现。

多模态记忆系统

多模态大模型的兴起推动记忆系统向多模态、跨模态方向发展，要求存储具备跨模态关联与毫秒级响应能力。

参数化记忆 (Model 层集成记忆)

在 Transformer 架构中引入可学习的记忆单元 Memory Adapter，实现模型层面原生支持用户维度的记忆。优点是响应速度快，但面临“灾难性遗忘”和更新成本高的挑战。

5.1.3 当前主要的技术路径

- 1. 外部记忆增强（当前主流）：使用向量数据库等外部存储来记忆历史信息，并在需要时通过检索相关信息注入当前对话。这种方式灵活高效，检索的准确性是关键。
- 2. 参数化记忆（深度内化）：直接将知识编码进模型的参数中。这可以通过模型微调、知识编辑等技术实现，优点是响应速度快，但面临“灾难性遗忘”和更新成本高的挑战。

5.2 相关开源产品对比

产品	开源时间	功能（记忆类型）	LLM的作用	向量数据库	图数据库
Mem0	2023年7月	✅ 用户画像+领域记忆	记忆创建/更新/检索	✅ pgvector, Qdrant等多种	✅ Neo4j, Memgraph
Zep	2023年初	✅ 用户画像+领域记忆	记忆提取+实体识别	✅ 支持	✅ 时序知识图谱架构
MemOS	2025年7月	✅ 用户画像+领域记忆	记忆提取+推理	⚠️ 可选支持，非必需	✅ NebulaGraph, Neo4j
Memobase	2025年1月	✅ 用户画像记忆 ❌ 领域记忆	画像理解+更新	❌ 不依赖向量数据库	❌ 不依赖图数据库
AgentScope ReMe	2025年6月	✅ 用户画像+领域记忆	记忆管理+优化	✅ Chroma, Qdrant等多种	❌ 未明确提及

关于各产品的具体数据指标对比，评测方式各有侧重，因此评测结果不尽相同，从实际情况看，各方均以 mem0 为评测基准，从各类技术指标评测结果以及开源社区的活跃度（star，issues 等）方面，mem0 仍然是占据长期记忆产品的领头地位。

结语

Cloud Native

记忆系统作为 AI Agent 的核心基础设施，其发展直接影响着智能体的能力和用户体验。现在各框架内置的压缩、卸载、摘要等策略，已经能解决 80-90% 的通用场景问题，但对于特定行业或场景，比如医疗、法律、金融等领域，基于通用的上下文处理策略基础之上进行针对性的处理和更精细的压缩 prompt 设计，仍然有较大的优化空间。而长期记忆作为可独立演进的组件，未来会更加贴近人脑的记忆演化模式，包括记忆的巩固、强化、遗忘等全生命周期管理，同时长期记忆应该以云服务模式提供通用的记忆服务，共同助力 Agent 迈向更高阶的智能。

相关阅读：

- 《AgentScope Java 答疑时间：开发者近期最关心的 12 个问题》
- 《AgentScope x RocketMQ：打造企业级高可靠 A2A 智能体通信基座》
- 《AgentScope Java v1.0 发布，让 Java 开发者轻松构建企业级 Agentic 应用》

参考文档：

[1] FlowLLM Context Engineering

https://github.com/FlowLLM-AI/flowllm/tree/main/docs/zh/reading

[2] Google ADK Memory

https://google.github.io/adk-docs/sessions/memory/

[3] LangChain Memory

<https://docs.langchain.com/oss/python/langchain/long-term-memory>

[4] AgentScope Memory

https://doc.agentscope.io/zh_CN/tutorial/task_memory.html

[5] O-MEM

<https://arxiv.org/abs/2511.13593>